



IIT Madras | BS Degree Programme

Data Science & AI

Course-Aware Personalized Learning Companion

for IIT Madras BS Degree Students

Milestone 3 Report | Retrieval-Augmented Generation (RAG) System

Group 3: Mayank Singh | Ali Jawad | Sachi Dhaturaha | Aryan Pratap Maurya | Jibin V Mathews

Objective of Milestone 3

1

Load Processed Course Chunks

Bring structured, cleaned content from Milestone 2 into the pipeline.

2

Generate Embeddings

Convert text chunks into dense vector representations for semantic search.

3

Index in Vector Database

Store embeddings in Qdrant for fast, hybrid retrieval at query time.

4

Retrieve Relevant Context

Identify the most relevant course chunks for any given student query.

5

Grounded Prompt Construction

Pass only retrieved context to the LLM, avoiding hallucinated answers.

6

Produce Faithful Answers

Generate responses traceable to specific course-material source chunks.

Dataset Organisation & Input Format

Directory Layout

```
data/  
  raw/  
    faq/   kartik_sir_notes/  
    MLT Weekly Notes/  pq/   PYQ/  
    transcripts/  
  processed/  cleaned/  
  splits/  
    train_chunks.jsonl  
    val_chunks.jsonl  
    test_chunks.jsonl
```

Week-wise Split Strategy

Train Split

Weeks 1 to 8

Core retrieval tuning content

Validation Split

Weeks 9 to 10

Structural separation, reduce leakage

Test Split

Weeks 11 to 12

Unseen content for final evaluation

Chunking & Retrieval Parameters

Chunk Size Limit

Max ~384 tokens per chunk with a small overlap. Keeps each retrieved unit compact and semantically focused, preventing dilution of meaning across large text spans.

Context Window Budgeting

Top-k is set to ~5 chunks per query. Limits retrieved context while providing enough evidence for a grounded answer, staying within practical LLM context limits.

IF

Input Shape for Processing

The embedding model receives the raw chunk text string as input; the retriever then passes a shortlist of matched chunk texts to the LLM. This two-stage handoff keeps the retrieval stage efficient and the generation stage controlled and compact.

MD

Metadata Fields per Chunk

Each chunk carries week number, source type, document ID, chunk ID, and related traceability fields. These enable filtered, course-aware search that goes beyond pure semantic similarity, allowing retrieval to be both relevant and contextually scoped.

Preprocessing Pipeline

01 PDF to Markdown

Raw PDFs and text files converted using PyMuPDF and PyMuPDF4LLM into normalised Markdown format.

02 OCR Fallback

EasyOCR processes scanned PYQ content and mathematical material where standard extraction fails.

03 FAQ Normalisation

HTML-based FAQ content is parsed and converted into structured text and Markdown-style layout.

04 Transcript Cleaning

Timestamp artifacts, boilerplate noise, and repeated headers are removed from all transcript files.

05 Chunk Creation

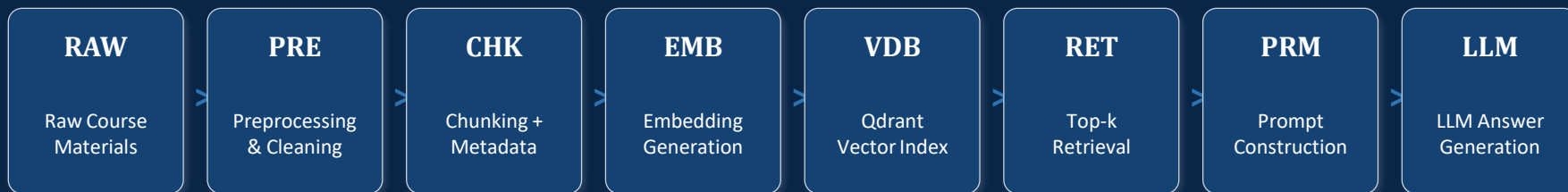
Cleaned text is split into retrieval-ready chunks with metadata: week, source type, document ID, chunk index.

06 JSONL Output

Final chunks stored as LangChain Documents in JSONL splits containing page_content and metadata fields.

Key Scripts: `process_dataset.py` `clean_dataset.py` `prepare_rag_splits.py`

System Architecture



Why RAG? Comparison with Alternatives

vs Fine-Tuning

FT

Requires labelled task data and a retraining cycle. RAG stays grounded without retraining and updates simply by re-indexing new course content, making it far more practical to maintain.

vs Keyword-only

KW

Fails on paraphrased or conceptual queries. Hybrid retrieval adds semantic matching on top of lexical matching, which is critical when students phrase the same concept differently.

vs Long-context LLMs

LC

Passing the full corpus raises cost, latency, and noise. RAG keeps the context compact, focused, and traceable by supplying only the top-retrieved chunks to the generation step.

vs Rule-based QA

RB

Static and hard to scale when course material grows. RAG adapts naturally to new documents via re-indexing, keeping answers grounded in the current indexed corpus without manual rule updates.

RAG offers the best balance of grounding, accuracy, explainability, and low implementation cost while remaining flexible for future extension.

Embedding & Retrieval Module

Embedding Pipeline

- 1 Load chunks from JSONL split files
- 2 Generate dense embeddings using FastEmbed
- 3 Generate sparse vectors for keyword-aware search
- 4 Index all chunks into Qdrant vector store

Model: BAAI/bge-small-en-v1.5 |
Dimensions: 384

Retrieval Behaviour

Dense Retrieval

Semantic similarity search using BAAI/bge-small-en-v1.5 embeddings at 384 dimensions. Captures conceptual meaning beyond exact keyword matches.

Sparse Retrieval

BM25-style keyword matching via Qdrant's built-in sparse vectors. Ensures course-specific terminology is matched precisely.

Hybrid Mode

Dense and sparse signals are combined to make retrieval robust for both conceptual questions and technical term lookups.

Top-k Output

Retriever returns the most relevant chunks to form a compact, focused context window before LLM generation.

LLM Integration & Prompt Engineering

Prompt Design Principles



Course-Grounded Only

Model is instructed to answer solely from retrieved course context, not from general training knowledge.



Hallucination Prevention

Outside knowledge and speculative claims are explicitly prohibited in the system prompt.



Descriptive Responses

The model is directed to produce approximately 200-word continuous answers rather than bullet lists.



Traceability Required

Model must cite the model name and reference the chunks it used to construct its answer.

LLM Connectivity

Supported Backends

Gemini (default) | Groq-backed LLMs

Both share the same interface via rag_pipeline.py

Structured Output Format



Direct Answer

A concise, grounded answer stated upfront, drawn solely from retrieved course context.



Explanation

A fuller elaboration of the concept, walking through the reasoning in the student's course terms.



Math Formula

Relevant mathematical expression or notation, included only where the concept requires it.



Example

A concrete illustrative example to reinforce understanding, grounded in course material.

Math formula step is included only when the question requires it.

End-to-End Verification & Evaluation Results

Verification Evidence

- ✓ Prompt-building pipeline imported and executed locally
- ✓ Sample grounded prompt generated (length: 1,151 characters)
- ✓ Chunking and retrieval modules confirmed in repository
- ✓ Retrieval and generation modules connected via shared interface
- ✓ Live LLM call requires active API key (documented limitation)

Evaluation Results (6 Sample Questions)

Topic	Faith.	Relev.	Prec.	Recall
Supervised vs Unsupervised	High	High	Med	High
Training Dataset	High	High	High	High
Overfitting	High	High	High	High
Underfitting	High	High	High	High
Bias-Variance Tradeoff	High	High	Med	High
Linear Regression Use	High	High	Med	Med

Best results on clearly-defined concepts (Overfitting, Underfitting). Context precision dips where retrieved chunks only partially cover broader questions.

Strengths & Limitations

Strengths

M

Modular Architecture

Each pipeline stage is independently implemented and can be extended or swapped without disrupting the rest of the system.

H

Hybrid Retrieval

Combining dense semantic and sparse keyword signals improves robustness over purely embedding-based or purely keyword-based search.

G

Grounded Prompting

The prompt design actively discourages the model from going beyond retrieved context, increasing answer reliability.

L

Local Compatibility

No supervised training is required; the system runs in a local development environment with standard tooling.

Limitations & Next Steps

!

API Dependency

Full LLM generation requires a valid Gemini or Groq API key; local-only evaluation is currently limited.

!

Retrieval Coverage

Answer quality depends on how well the underlying course chunks cover the topic being asked about.

!

Noisy Metadata

Some week-level labels in chunk metadata remain imprecise and should be further cleaned in future iterations.

!

Small Evaluation Set

Current evaluation covers only 6 questions; a broader test set is needed to validate system quality at scale.

Individual Contributions

AJ

Ali Jawad

22f3001825

Embedding Pipeline

Developed the full embedding pipeline: loading processed document chunks, generating embeddings using BAAI/bge-small-en-v1.5, storing vectors in Qdrant, and documenting model details including dimensions and chunk size configuration.

MS

Mayank Singh

23f1000598

Retrieval Module

Set up the Qdrant vector database, indexed all document embeddings, designed and executed similarity search, and implemented Top-k chunk retrieval for downstream generation.

SD

Sachi Dhaturaha

21f1000471

LLM & Prompt Engineering

Designed and implemented the LLM integration and prompt engineering workflow, including prompt templates, LLM connectivity via Gemini/Groq, context feeding, and final response generation.

JM

Jibin V Mathews

21f1001895

Evaluation & Testing

Created sample student queries, ran the complete RAG pipeline end-to-end, recorded retrieved chunks and generated responses, and analysed performance across faithfulness, relevance, precision, and recall metrics.

AM

Aaryan Pratap Maurya

22f1000559

Architecture & Report

Designed the overall system architecture, created workflow diagrams, and compiled the full Milestone 3 report, ensuring completeness, clarity, and proper documentation of all deliverables.